Data Science Team Collaboration Guidelines

The contents of this file have been migrated to the nuclino team documentation

This document attempts to outline some best practices for the SPGMI Data Science team as far as collaboration goes, to optimize our functioning as a team.

Projects

It's very important that we be able to share and reproduce our results with other team members and people outside the group — that's critical for any scientific endeavor, data science included.

Project Guidelines

To that end, there are some important guidelines around projects that everyone must follow:

- The preferred medium for our work are Jupyter notebooks (and related files) that are part of a Domino project. This includes both notebooks that run in Domino and ones that must run on local machines (for example, Hadoop clusters).

- The structure of projects should follow the standard template, and include an appropriate level of description in the top-level README.md file to allow a team member to orient him/herself to the project quickly.
  - Detail on the project template is included below.
  - Legacy SNL DRA models that we've built under the phase 1 PRB are grandfathered as-is.

- Python code should follow the standard Python style guide.

- If a project is capable of being run in the Domino cloud, it should run properly in the Domino cloud. The only projects that are not expected to run in the Domino cloud are ones where there is an explicit dependency on some local resources, such as Hadoop.

- Projects should be as self-contained as possible. Specifically, one should be able to run and evaluate models within the notebook without require access to any resources (files, calculations, etc.) that occur outside the project.

- In cases where data is brought into the project manually (i.e. placing .CSV files into the data folder), document the root source of the data appropriately (the README.md could be a good place for this), so that the original source data can be found if needed.

- Notebooks should always be in a state that facilitates sharing, even for works in progress. This includes:

- Notebooks should include an appropriate level commenting, via Markdown cells and inline comments. Don't assume that the reader is intimately familiar with the process; walk him/her through what you're doing.

- The notebooks in Domino should be the current, up-to-date versions.

- Any graphs or other generated files should be stored in the project and, ideally, inlined into the saved versions of the projects/notebooks. Specifically, one should not have to run notebook code to generate the visualizations.

- Key visualisations or other important generated files should be written to the output folder of the project so that they will appear as Results in the Domino UI.

- When sync'ing projects from the Domino cloud to a local workstation or development server, one should follow the convention below for storing projects on the local file system. This will help keep the directory structure consistent between how projects are stored in the Domino cloud and on various local machines.

- Users should have a single root Domino folder on the local machine, e.g. **~/Domino**

- The root Domino folder should contain a series of folders representing the usernames of Domino project owners, including both Organization and Personal usernames e.g.:
  - **~/Domino/SPGMI-dev/**
  - **~/Domino/SPGMI-poc/**
  - **~/Domino/GalenWarren/**

- When downloading a project from the server or creating a new project locally using the Domino CLI, the project should be placed into the folder that corresponds to the Domino username of the project owner. Thus, the path for the project folder (starting from the root Domino folder) becomes **[Domino Owner]/[Project Name]**, the same as in Domino cloud.

- The root Domino folder should not contain any "orphaned" project folders. All project folders must be filed in a directory corresponding to their Domino project owner.

Project Collaboration Workflow with Fork-and-Review

For each project, there is one "master" project, which at every given time holds the current "gold standard" snapshot. This master project lives on Domino and belongs to one of the company's Organization accounts, such as SPGMI-dev or SPGMI-poc.

The source code in the master project is never modified directly. Each data scientist working on the project, including the lead, fork a private copy into their own Domino account at [Personal User]/[Project Name], and work on that code base exclusively. The entire workflow to start a generic new project looks like this:

1. Create the master project at [Organization Name]/[Project Name], following the Project Template guidelines as described below.
2. Click the "Fork" button in the master project to create a personal fork at [Personal User]/[Project Name].
3. Download fork to local machine for development (or work on Domino).
4. Back up to Domino at least daily; syncs do not affect master project.
5. When something shareable is ready, submit a review request by clicking the "Request Review" button in the personal fork to merge work into master.

When sufficient progress has been made on the private fork, a "Review Request" is submitted, which is reviewed by the one of the data scientists involved with the project before accepting. The pending review request can be a good opportunity for peer code review, but doesn't have to be. The request should have a title that distinguishes the work done from other merges, and a description that summarizes the results of the work.

It is the project lead's responsibility to update the master project regularly, including before and after code review, before regular project meetings and other milestones like Sprint Review.

The benefits of this workflow are:

- Changes to various files are globbed into a descriptive update for others to read.
- The project is tracked chronologically so someone joining in or catching up can review.
- Supporting data scientists on the project do not need to rely on external communication (Slack, email) to submit changes to a project.
- Comments, questions and discussion can happen in the “Discussion” tab of the “Reviews” page, which everyone can see and is permanently associated with the project.

Project Template Detail

The standard Domino project template looks like this:

```
├── [project root]/
|   ├── data/
|   ├── executable/
|   |   ├── lib/
|   ├── intermediate/
|   ├── output/
|   ├── reference/
|   ├── scratch/
|   ├── .dominoignore
|   ├── .dominoresults
|   ├── .dominostats
|   ├── README.md
└──── requirements.txt
```

Links are included to relevant Domino documentation where applicable. Each sub-folder also contains its own README.md, which have been omitted.

To set up new Domino project with the project template, you can use this process:

- Use the Domino CLI to get an up-to-date copy of the **ProjectTemplate** project to your local machine:
  - **domino get SNL/ProjectTemplate** ... (if you are downloading the template for the first time)
  - **domino download SNL/ProjectTemplate** ... (if you already have a copy).
- Create a folder for the new project in the appropriate spot in your local Domino directory, e.g. **~/Domino/[Domino Owner]/[New Project Name]**
- Copy the contents of the **ProjectTemplate** folder to the new project folder
- Delete the hidden **.domino** directory in the new project folder
- In a terminal window, change directories to the new project folder, then use the Domino CLI to initiate the folder as a Domino project: **domino init --owner [Domino Owner]**

A brief description about the parts of the template and how they should be used is below.

README.md

The README.md file in a Domino project's root folder serves as the project's "Overview" pane in the Domino Web UI. It's the first stop for team members who aren't familiar with the project to get oriented to the project's high-level goals and approach.

The project template contains an example project overview. The Data Science Lead for each project should fill out the README.md file, and keep it up to date as the project evolves.

data

Looking in the data folder should clearly answer the question "what is the complete set of raw source data used for this project?"

It should contain any project files related to original source data. This may include:

- Data files, such as .csv and .json
- SQL queries

- Other files related to connecting to data sources

The data folder should *only* include original source data. The data folder should *not* include data that is not raw source data, e.g. intermediate, transformed datasets or final output data. These belong in the intermediate and output folders, respectively. Following the convention is very important; otherwise new data scientists joining the project will not know what is and isn't original source data.

executable

Looking in the executable folder should clearly answer the question "what is the complete set of code required to generate the project's output, given the raw source data in the data folder?"

It should contain Python scripts and Jupyter Notebooks that are intended to be executed as the project's primary workflow. They should be in a runnable state and reasonably documented.

The executable folder contains a lib sub-folder. Scripts and notebooks that contain functions and classes primarily intended to be imported in other project components, but not run directly themselves, should live in the lib directory.

The executable folder should not contain purely exploratory code, components in an early stage of development, or anything else that isn't on the project's critical path to move from the raw data to the output.

intermediate

The intermediate folder should contain saved Python objects and intermediate, transformed data, e.g.:

- MySVM.pkl
- ModelTfidfTransformer.pkl
- transformed_dataset.csv

If the intermediate folder becomes cluttered, it's fine to create sub-folders to further organize the components, e.g.:

- **intermediate/data**
- **intermediate/models**

output

The output folder should contain any output files produced by the project's workflow. If some set of these files should be considered experimental results and receive special attention (e.g. summary statistics, performance metrics, plots), please add the filename(s) to the project's **.dominoresults** file.

reference

The reference folder should contain any reference material required to understand the project or its modeling approach, e.g. technical papers published as PDF files.

scratch

Use the scratch folder for any code that doesn't belong in executable. This might include components that are in an early state of development, or are purely exploratory or experimental in nature.

By following these guidelines, we should end up with a set of projects and notebooks that we can freely share and easily collaborate on, which will help improve the quality and quantity of output of the team.

Communication

Another key element of successful team collaboration is communication. Projects are generally assigned to individuals but ultimately the responsibility for our work lies at the team level, and we will frequently need to coordinate on projects. Also, we must communicate effectively with people outside the team to be successful.

Communication Guidelines

So, some guidelines for communication:

- If you need something from someone, or are unclear on what next steps in a project should be, or really if you have any question at all -- speak up and ask it! It's very important that we have a culture of open communication and discussion so that we can avoid any wasted effort or backtracking.

- A corollary to this is that it's very important for everyone to be acutely aware of when you are confidently on the right track or not for a user story or task. If anyone ever has any questions about those sorts of things, please speak up immediately so we can settle any open questions.

- In general, be as responsive as possible, even if just to say that a message was received and you'll get back in some period of time. Dead air time between messages can really kill productivity.
- Related, we use Slack for messaging. Whenever possible, please be logged in here so you can be reached if needed.

- If you need something from someone, be persistent in requesting it -- don't be stuck waiting for a response. Specifically, if you need something and you've asked for but haven't received any sort of reply, ask again -- don't assume that your request was received. Questions get missed sometimes!

- The more concise and specific communication can be, the better. For example:

- When asking questions, make sure the questions are clear and called out if part of a larger message. Wherever possible, make sure you indicate who the question is directed towards - and include that person's name. Consider bolding it in the request if you can. Etc.

- When reporting results, include an executive summary of the important key results and interpretations before diving into the gory details. That can help quickly orient people to what you're trying to communicate. And ideally any actual data or charts would be links to files in a Domino project (notebook or other files), not inline in email or other messages.

Domino Communication Features

The Domino platform provides some specific features that can assist with intra-team communication and collaboration:

- Users can give custom names to runs in a Domino project. When conducting a run, users should give the run a concise but descriptive name about the purpose of the run.

- Domino allows two types of "comments" in the platform:

  - Comments on runs are available on a project's "Runs" page, by clicking on a particular run record, then on the "Discussion" tab. Comments on runs stay with the run record, so will become less prominent as new runs occur and appear higher in the project's runs list. They are most useful for timely questions or commentary about the unique attributes of a particular run.
  - Comments on files are available through a project's "Files" page, by clicking into a particular file. Comments on files stay with the file for the lifetime of the file, or until the comment is deleted/archived. They are most useful for general questions and commentary about project data, modeling strategy, coding tactics, etc., expressed in project files.

- Both types of comments are aggregated and displayed in a project's "Discussion" page from the left-side navigation bar.

- Domino comments support @-mention notifications using team members' usernames. When a comment is intended for one or more other team members, @-mention their Domino username(s), so they are notified.

By following these guidelines, we should be able to harness the collective knowledge and expertise of the team in the most efficient way possible and also avoid wasted effort.